Ders 7 Genişletilmiş Notu: Sistem Tasarımı: Ön Tasarım, Altyapı Kararları, Girdi-Çıktı-Ara Birim Tasarımı ve Yapı Diyagramı

Ders 7 Genişletilmiş Notu: Sistem Tasarımı: Ön Tasarım, Altyapı Kararları, Girdi-Çıktı-Ara Birim Tasarımı ve Yapı Diyagramı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Buraya Kadar Ne Yaptık, Şimdi Nereye Geçiyoruz?
- 2. Sistem Tasarımının İki Safhası
- 3. Bir Bilgi Sisteminin Tasarlanması Gereken Parçaları
- 4. Tasarım Sürecinin Akışı
- 5. Sistem Tasarımı Yaklaşımları
- 6. Ön Tasarım: Çözüm Nasıl Temin Edilecek?
- 7. İşlevsel Olmayan Gereksinimler ve Mimari Sorular
- 8. Mimari Tasarım Bileşenleri
- 9. Envanter, İş Yükü, Değerlendirme ve Satıcı Seçimi
- 10. Edinme Biçimi: Satın Alma, Leasing, Kiralama
- 11. Ayrıntılı Tasarım ve Çıktı Tasarımı
- 12. Girdi Tasarımı
- 13. Ara Birim Tasarımı
- 14. Yazılım Tasarımı ve Modüler Program Yapısı
- 15. Yapı Diyagramının Öğeleri
- 16. Yapı Diyagramı Örnekleri
- 17. Tasarım Belirtimleri
- 18. Modüllerin İçini Göstermek: IPO Kartları ve HIPO
- 19. Bugün Uygulamada Ne Yapılıyor?

1. Buraya Kadar Ne Yaptık, Şimdi Nereye Geçiyoruz?

1.1 Önceki aşamaların hatırlatması

Bu haftaya kadar sistem geliştirmenin ilk aşamaları geçildi. Önce bir ön inceleme yapıldı ve bunun çıktısı olarak project charter denen doküman üretildi. Sonra fizibilite çalışması ve onun sonucu olan fizibilite raporu geldi. Ardından iki kısımdan oluşan sistem analizi safhası işlendi: iş analizi ile işlevsel ya da nesneye yönelik çözümleme. Bu safhanın sonunda iki temel doküman üretildiği hatırlatıldı. Bunlardan biri yazılım gereksinim belirtimi, yani SRS (Software Requirement Specification) dokümanıdır.

Bu aşamalar sayesinde iki grup soru yanıtlandı. Fizibilitede problemin ne olduğu ve çözüm için nasıl bir zaman planına, maddi kaynağa, iş gücüne, makine ve teçhizata ihtiyaç duyulduğu belirlendi. Analizde ise sistemin işler hale gelmesi için nelerin yapılması gerektiği, işlerin bugün nasıl yürüdüğü ve bunların bir bilgi sistemine, yani yazılım ortamına nasıl aktarılacağı ortaya çıkarıldı.

1.2 “Ne” sorusundan “nasıl” sorusuna

Sistem tasarımında soru değişir. Artık gereksinimlerin ne olduğu sorulmaz. Bu gereksinimlerin nasıl yerine getirileceği, çözümün nasıl olacağı ve kodlanacak sistemin nasıl bir yapıda olacağı sorulur. Tasarım aşaması bu kararların ortaya çıkarıldığı ve dokümante edildiği aşamadır.

Tasarımın alt aşamaları şöyle sıralanır: kullanılacak altyapının belirlenmesi ve genel tasarım, yani ön tasarım; ardından ayrıntılı tasarım. Ayrıntılı tasarımda önce sistemin daha basit tarafları olan girdi, çıktı ve ara birim tasarımı ele alınır. Asıl ağırlık ise yazılım tasarımındadır. Veri tasarımı da bu aşamanın parçasıdır, ama derste ayrı bir hafta olarak bütün halinde işlenmek üzere bırakılmıştır.

Bu haftanın sorusu şudur: işlevsel modelleme ile yola çıkıldıysa nasıl bir tasarım yolu izlenir, hangi çıktılar üretilir, bunlar nasıl oluşturulur? Bir sonraki hafta aynı soru nesneye yönelik taraf için sorulacaktır: senaryolardan çıkarılan kavramsal sınıf diyagramı kullanılarak nesneye yönelik bir yazılım nasıl tasarlanır?

Genel ve ayrıntılı tasarım tamamlandığında ortaya bir model, bir çözüm önerisi ya da çözüm taslağı çıkar. Buna **tasarım modeli** adı verilir.

2. Sistem Tasarımının İki Safhası

Sistem tasarımı iki ana safhadan oluşur: ön tasarım ve ayrıntılı tasarım.

Ön tasarımda önce altyapı belirlenir. Sistem bir donanım altyapısında çalışacaktır. Bu altyapı fiziksel olarak kurumun kendi tarafında barındırılabilir, kiralanabilir ya da fiziksel temasın hiç olmadığı bir bulut ortamında bile bulunabilir. Bu seçeneklerin artı ve eksileri tartışılır ve kurum için doğru olan altyapı kesinleştirilir.

Burada akla şu soru gelir: bunlar fizibilitede yapılmamış mıydı? Yapılmıştı, ama fizibilitedeki kararlar öneri ve tavsiye niteliğindeydi. Tasarıma geçildiğinde bu seçimler artık gerçek hayatta birebir uygulanacak biçimde kesinleşir.

Ön tasarımda ikinci iş, kabaca modül mimarisini oluşturmaktır. İşlevsel çözümlemeden gelindiğinde şu sorular sorulur: yazılımın parçaları neler olacak, işlevsel modüller neler olacak? Veri akış diyagramlarındaki prosesler birer modül mü olacak, yoksa bir modül ve onun alt modülleri gibi bir yapı mı kurulacak? Bunların çalışması için hangi kontrol ve veri mesajları gidip gelecek?

> **Ders sonrası not:** Altyazıda veri akış diyagramları için “DFT” kısaltması geçiyor. Doğru kısaltma DFD’dir (Data Flow Diagram).

Ayrıntılı tasarımda etkileşim tasarımı yapılır, yani girdi, çıktı ve ara birim tasarımı. En önemli kısım ise yazılımın, yani programın tasarımıdır: kodlama tarafındaki işlerin nasıl yapılacağı burada belirlenir. Bunlarla eş zamanlı yürütülen bir diğer alt adım veri tasarımıdır: veri yapısı tasarımı ve verinin fiziksel olarak tutulması için veri tabanı tasarımı.

Özetle ön tasarımda altyapı ve modüler çözümleme ortaya konur. Ayrıntılı tasarımda girdi, çıktı ve ara birim tasarımı, yazılım tasarımı ve veri tasarımı tamamlanır ve ifade edilir.

3. Bir Bilgi Sisteminin Tasarlanması Gereken Parçaları

Slayttaki şekilde, günümüzde standart sayılabilecek bir bilgi sisteminin parçaları görülür. Hocanın dikkat çektiği nokta şudur: “sistem budur” diye gösterilebilecek tek bir yer yoktur. Sistem bu parçaların neredeyse hepsinden oluşur. Her parçanın tasarlanması ve parçaların bir arada nasıl çalışacağının da ortaya çıkarılması gerekir.

Uygulama sunucusu ve uygulama tasarımı. Senaryolarda anlatılan işleri yerine getirecek kodlar ve programlar uygulama sunucusunda durur. Yazılım tasarımı derken kastedilen, bu sunucu altında duran uygulamaların tasarlanmasıdır.

Veri tabanı sunucusu ve veri tabanı tasarımı. Uygulamalar girdi alıp çıktı üretir, yani input, process, output zincirini işletir. Bunun için verilerin bir yerde durması gerekir. Bu nedenle ayrı bir veri tabanı sunucusu yapısı olur.

Şekilde bu sunucular şirket içinde, aynı alt ağda, yerel ağ bağlantılarıyla gösterilmiştir. Bu tek doğru çizim değildir. Veri tabanı sunucusu da uygulama sunucusu da internet üzerinde bambaşka yerlerde duruyor olabilir. Çok farklı kombinasyonlar yaratılabilir.

Uygulama sunucusu ile veri tabanı sunucusu aynı fiziksel makine üzerinde de durabilir. Ama bunlar farklı yazılım sunucularıdır. Bu yüzden genellikle bağımsız, ayrı makineler olarak temsil edilirler.

Erişim noktaları ve kullanıcı arayüzü tasarımı. Şekildeki ofis bilgisayarları, masaüstü ve dizüstü bilgisayarlar, veriyi kullanan uygulamaların erişim noktalarıdır. Bu uç noktalardan bilgi sistemine erişip onu kullanmak için kullanıcı arayüzlerine ihtiyaç vardır. Ekranlar, üretilen raporlar, girdiler, çıktılar ve bunlar üzerinde yapılması gereken kontroller, girdi ve çıktı tasarımını ya da hepsi dahil olmak üzere kullanıcı arayüzü tasarımını oluşturur.

Kapalı bir sistemde, yani yalnız kendi içimizde kullanılan bir sistemde erişim noktaları, uygulama sunucusu, veri tabanı sunucusu ve bunları bağlayan ağ temelde yeterlidir.

Başka sistemler ve sistem arayüzü tasarımı. Sistem tek başına çalışmıyorsa, başka sistemlerle entegre olup veri alışverişi yapması, onlardan bilgi toplaması ya da onlara bilgi sunması gerekiyorsa, onlarla konuşması gerekir. Diğer sistemlerle haberleşmeyi tanımlayan arayüzlerin de tasarlanması gerekir.

Hocanın örneği şöyledir. Sistemleriniz Windows işletim sistemine sahip ve üzerinde IIS kullanan web tabanlı bir uygulama çalışıyor. Bir kullanıcıyı yetkilendirmek için onun Türkiye Cumhuriyeti vatandaşı olup olmadığını kontrol etmeniz gerekiyor. MERNİS sistemine bir kimlik numarası verip "bu numara doğru mu, böyle bir vatandaş var mı?" diye soruyor ve evet ya da hayır cevabı alıyorsunuz. Karşıdaki sistem belki de Linux üzerinde Java ile yazılmış, bambaşka bir altyapıya sahiptir. İki taraf nasıl konuşacak? Bunun tasarlanması ve belli protokollere göre konuşulması gerekir. Bunların hepsi sistem arayüzü tasarımının içindedir.

Güvenlik ve kontrol tasarımı. Uygulama sunucusu, veri tabanı sunucusu, hizmet alınan ve verilen başka sistemler ve erişim uç noktaları internetin farklı yerlerindeyse, bilgi sisteminin çalıştığı ortamın izole edilmesi, korunaklı hale getirilmesi gerekir. Şekilde bu, tuğla duvar biçiminde bir güvenlik duvarı ile gösterilmiştir. Ama konu bir güvenlik duvarından çok daha geniştir: uygulama güvenliği, ağ güvenliği, erişim güvenliği, kullanıcı güvenliği ve veri güvenliği gibi alt dallara ayrılabilir. Bu nedenle güvenlik tasarımı da ayrı bir tasarım alt başlığıdır.

> **Hocanın vurgusu:** Bir sistemi tasarlamak, "modüllerimiz bunlar olsun, Java kullanalım, şu kütüphanelerden faydalanalım, ekranları Swing ile yazalım, Hibernate kullanalım" demek değildir. Erişim uçları, uygulama sunucusundaki yazılım, veri, güvenlik ve başka sistemlerle haberleşme ayrı ayrı tasarlanmalıdır. Ancak bunların hepsi tamamlandığında bir bilgi sisteminin tasarlandığı söylenebilir.

4. Tasarım Sürecinin Akışı

Tasarımın aşamaları kitaba uygun olarak bir şekille de gösterilir. Akış şöyledir:

1. Bir önceki aşama olan gereksinim analizinden sistem gereksinimleri gelir.
2. Bu gereksinimler üzerinde çalışılarak ön tasarım yapılır: altyapı, modüler ayrıştırma, sistemle ilgili genel kısıtlar ve modüllerin çalışacağı temel veri ve kontrol mesajları belirlenir.
3. Bunlar bir ön tasarım raporu ile ifade edilir ve idari incelemeden geçer. Bu inceleme fizibilite ve gereksinim analizi sonrasında da yapılmıştı.
4. Gerekiyorsa revize edilir ve tekrar incelenir. Ön tasarım kabul edilince ayrıntılı tasarıma geçilir.
5. Ayrıntılı tasarımda her modül, arayüzler, veri tabanı, güvenlik, diğer sistemlerle etkileşim arayüzleri ve yazılım tasarımı yapılır. Bu çalışmayı ön tasarım destekler.
6. Ayrıntılı tasarım da incelenir, gerekirse revize edilir.
7. Kabul edildiğinde tasarlanmış bir sistem çözümü elde edilmiş olur. Çözüm, gerçekleştirme aşamasına taşınır.

5. Sistem Tasarımı Yaklaşımları

Bir sistem tasarımı oluşturmanın tek bir yolu yoktur. Farklı çözüm yaklaşımlarına **sistem tasarımı yaklaşımları** denir. Derste en çok kullanılan üçü anlatıldı. Geçmişte daha çok kullanılan başkaları da vardır, ama bunlar kapsama alınmadı.

5.1 Model güdümlü yaklaşım

Model driven denen yaklaşımda elimizde analiz modelleri, yani mantıksal modeller vardır. Bunları biz yapmış olabiliriz ya da yalnız tasarım ekibi olarak hazır almış olabiliriz. Bu analiz modeli kullanılarak çeşitli tasarım modelleri oluşturulur. Mimarlık ve başka mühendislik dallarında da benzer tasarım çalışmaları yapılır.

Çeşitli adaylar üretilir: "Bu gereksinimleri karşılayan şöyle bir sistem çözümü mü geliştirsek, böyle bir çözüm mü?" Bunlar prototip gibidir, ama aslında üretilmez, tartışılır. "Şu teknolojilerle şöyle bir çözüm üretsek ne olur? Bunu karşılar ama şunu karşılamaz, onun için şu kadar uğraşmamız gerekir." Artıları ve eksileri konuşulduktan sonra, gereksinimleri ve kalite beklentilerini en uygun biçimde karşılayan model çözüm adayı olarak alınır ve onun bütün yönleriyle ayrıntılı tasarımı yapılır.

Modern yapısal tasarım, bilgi mühendisliği ve prototipleme bu yaklaşımın alt kırılımlarıdır. Derste bunların ayrıntısına girilmedi. Hocaya göre kabaca ne yapıldığını bilmek yeterlidir.

Bu yaklaşım uzun bir süreçtir. Aday çözümlerin çoğunu, en azından birini çöpe atacaksınız, ama hepsinin ön tasarımını yapmak zorundasınız. Bu yüzden iş yükü fazladır. Çok garantili bir ürün elde etmeniz gerekiyorsa tercih edilebilir.

5.2 Hızlı uygulama geliştirme (RAD)

Rapid Application Development yaklaşımının çıkış noktası şudur: "Bu kadar önceden oturup düşünmeyelim, hızlıca uygulama geliştirelim." Burada da bir model hazırlanır, ama bu bir prototiptir. Başlangıç noktası genellikle çok kullanılan, revaçta olan ve benzer problemlerde herkesin kullandığı çözümdür.

Prototip sürekli gözden geçirilerek ve yeniden tasarlanarak iyileştirilir. Bunu yaparken bilgi sistemini geliştiren taraflarla ortak uygulama geliştirme oturumları yapılır. Hoca burada spiral modeli hatırlattı: önce analiz, sonra mühendislik faaliyeti. Modeli hazırlıyor, prototipi geliştiriyorsunuz. Sonra oturup konuşuyorsunuz: "Bu oldu mu? Nesi eksik? Ne ekleyebiliriz?" Paydaşların görüş ve katkılarıyla prototip bir model daha ileriye taşınır. Yani bir iterasyon daha yapılır, yeni modüller ve yeni özellikler eklenir. Örneğin önce yalnız çekirdek uygulama, sonra veri katmanı, sonra arayüz katmanları eklenerek adım adım ilerlenir.

5.3 Ortak uygulama geliştirme (JAD)

Bu yöntem genellikle çok büyük projelerde kullanılır. Tasarımla ilgili bütün paydaşlar bir araya gelir ve bir seminer ya da atölye çalışması yapılır. Bu birkaç gün sürebilir. Kocaman bir salonda, mesai yapar gibi sabah 8'den akşam 5'e kadar çalışılır. Konu yansıtılır ve tartışılır: önce genel altyapı nasıl olsun, sonra bu altyapı için modüller neler olsun, sonra her modül nasıl olsun. Girdileri, çıktıları, yazılım tasarımı, veri tasarımı, arayüzleri ve diğer sistemlerle haberleşmesi ele alınır.

Herkes bir arada olduğu için "bir dakika, bu beni engeller" ya da "bunu şöyle yaparsak daha iyi olur" gibi farklı bakış açıları hemen ortaya çıkar. Yaklaşımın ruhu "kim ne diyecekse şimdi söylesin" sözüyle özetlenebilir.

Gerekçesi şudur: Bir parçanın tasarımını tek başına yaparsanız, o tasarım çok iyi bile olsa sistemin genelinde başka kısımları bozuyor ya da kötü etkiliyor olabilir. Böyle durumlar oluşabilecekse bütün katılımcıların katkı sağladığı bu seanslar kullanılabilir.

5.4 Yaklaşımların karşılaştırılması

Ortak uygulama geliştirme pek sık yapılmaz, ancak çok büyük projelerde görülür. Günümüzde daha çok hızlı uygulama geliştirme ve türevleri kullanılır: zaten herkesin kullandığı bir mi-

mariyle başlanır, o mimarinin gerektirdikleri yapılır ve probleme uydurularak prototip ürüne doğru ilerletilir. Model güdümlü yaklaşımın iş yükü daha fazladır, ama garantili ürün gerektiğinde seçilebilir.

Yaklaşım	Nasıl çalışır?	Ne zaman?
Model güdümlü	Birkaç aday tasarım üretilir ve tartışılır, en uygunu ayrıntılı tasarlanır	Çok garantili ürün gerektiğinde; süreç uzun, iş yükü fazla
RAD	Prototip hazırlanır, oturumlarla iterasyon iterasyon geliştirilir	Günümüzde en yaygın
Ortak uygulama geliştirme	Bütün paydaşlar atölyede birlikte tasarlar	Yalnız çok büyük projelerde

6. Ön Tasarım: Çözüm Nasıl Temin Edilecek?

Ön tasarımın işi altyapıyı belirlemek ve modüler çözümleme yapmaktır. Önce fizibiliteden gelen öneriler değerlendirilir ve bunların nasıl temin edileceğine bakılır. Bu soru yazılım, donanım ve ağ altyapısı için geçerlidir. Seçenekler şunlardır: sıfırdan kurum içinde geliştirmek, hazır çözüm satın alıp özelleştirmek, işi dışarıya yaptırmak (outsource) ya da çalışan sistemi doğrudan servis olarak almak.

Kurum içi geliştirme. Ürün tamamen sizin olur ve size özeldir. Her yapısı istediğiniz gibi değiştirilebilir. Hoca bunu terziye kıyafet diktirmeye benzetti, hatta terzinin anneniz olduğu durum gibi: tamamen öz kaynaklarla yapılan bir iş. Fikri mülkiyet hakları dahil her şey size aittir. İyi tasarlandıysa belki pazarlanabilir ve başkalarının da kullanması sağlanabilir. Buna karşılık çok daha pahalıya mal olur: ham maddeyi alacaksınız, geliştirme için zaman harcayacaksınız, iş gücünü finanse edeceksiniz. Daha uzun sürebilir ve ek istihdam gerektirebilir.

Satın alıp özelleştirme. Satın alma bir maliyettir, ama toplam maliyet kurum içi geliştirmeye göre düşükse tercih edilebilir. Eksisi şudur: ürünün sahibi siz olmazsınız, fikri ve mülki haklar sizde olmaz. Ürün tamamen sizin ihtiyaçlarınız için geliştirilmediğinden ihtiyacınızı yüzde yüz karşılamayabilir. Çeşitli uyarlamalarla probleminize yaklaştırılır, ama birebir terzi işi bir çözüm çıkmayabilir.

Dış kaynak kullanımı (outsource). Burada fikri ve sınai haklar tartışılır. Eser sahipliği ve kullanım hakkı gibi hakların bir kısmı üretende, bir kısmı kullananda olur. Genellikle normal bir kurum içi geliştirmeden bile daha pahalıya mal olabilir. Karşılığında daha hızlı bir çözüm elde edebilirsiniz. Çözüm, hazır bir ürüne göre daha özelleştirilmiştir ve kendi yaptığınıza olabildiğince yakındır.

Hizmet olarak alma. Bilgi sisteminden beklentileriniz tamamen servis olarak alınır. Altyapıyla da geliştirmeyle de uğraşmazsınız. Ama bu, kira öder gibi pahalıya gelebilir: ilk yatırım maliyetiniz yoktur, ancak onun fazlasını zaman içindeki kullanım maliyetine yedirmiş olursunuz. “Birden çok para harcamayayım, hemen kullanmaya başlayayım, abonelik gibi kullanayım” diyorsanız ve bu işinizi çözüyorsa uygun bir seçenektir.

Seçenek	Artısı	Eksisi
Kurum içi geliştirme	Tamamen size özel, haklar sizde, belki pazarlanabilir	Çok daha pahalı, uzun sürebilir, ek istihdam gerekebilir
Satın alıp özelleştirme	Toplam maliyet daha düşük olabilir	Haklar sizde değil, ihtiyacı tam karşılamayabilir
Dış kaynak	Daha hızlı, kendi yapacağınıza yakın	Kurum içinden bile pahalı olabilir, haklar paylaşılır
Hizmet olarak alma	İlk yatırım yok, uğraş yok	Kira gibi; toplamda pahalıya gelir

Derste ağırlık kurum içinde ya da dış kaynakla geliştirilen çözümlerdedir. Hazır bir ürünü parasını verip kullanmak üzerinde pek durulmaz.

7. İşlevsel Olmayan Gereksinimler ve Mimari Sorular

> **Hocanın vurgusu:** Kurum içinde ya da para verip dışarıda geliştirin, bir sistemin mimari tasarımını en çok etkileyen nokta işlevsel olmayan gereksinimlerdir (non-functional requirements). Mimariyi ve tasarımı doğrudan belirleyen bunlardır.

İşlevsel olmayan gereksinimler use case olarak yazılmaz. Use case'lerin altındaki özel teknolojik beklentiler gibi kısımlarda yer alır. Hoca aradaki farkı şu örneklerle anlattı:

- **Süre:** Rapor üretmek bir şeydir, 1 milyon kayıttan oluşan bir raporu 100 milisaniyede üretebilmek başka bir şeydir. Süre kısıtı konduysa buna uygun tasarım yapmak çok zorlaşabilir.
- **Kapasite:** Ders seçme sisteminde öğrencilerin ders seçebilmesi ayrı bir şeydir. Saniyede 10 bin öğrencinin eş zamanlı bağlanabilmesi ve 80 bin ders kaydının bir saniye içinde yapılabilmesi başka bir şeydir.
- **Güvenlik doğrulaması:** İki seviyeli güvenlik doğrulamasına sahip olmak.
- **Zararlı yazılımlara karşı koruma:** İşlevsel gereksinimlerde yazmaz, ama ayrı tasarım ve ek iş gerektirir.

Bu gereksinimlerin yanı sıra donanım ve yazılım altyapısına yönelik pek çok kararın da burada verilmesi gerekir. Mimari modelleme sırasında yanıtlanması gereken sorular, büyük ölçüde işlevsel olmayan gereksinimlerden doğar:

- **Sistem merkezi mi, dağıtık mı olacak?** Bir yükü karşılamak ya da belli bir kapasiteye hizmet vermek gerekiyorsa bunu merkezi sistemle yapmak çok zordur. O zaman daha ilk aşamada dağıtık çalışacak şekilde tasarlamak gerekir: uygulama sunucusundaki modüller farklı fiziksel cihazlarda aynı anda çalışabilmelidir. Eş zamanlılık gibi problemler de yazılım tasarımına eklenir.
- **Veri tek merkezde mi, dağıtık mı duracak?** Pek çok modern veri tabanı verileri dağıtık ya da çok kopyalı tutabilir. Kopyalar nasıl birleştirilecek? Bir kayıt güncellendiğinde hepsinde güncellenecek mi, yoksa biri güncellenip diğerleri eski mi kalacak? Bunların kontrolü nasıl olacak?
- **Geliştirmede neler kullanılacak?** Uygulamalar, kütüphaneler, teknolojiler, programlama dili ve araçlar.
- **Hazır alınan parçalar nasıl birleştirilecek?** Örneğin muhasebe sistemi genellikle yazılmaz, hazır alınıp entegre edilir. O sistem bize uyumlu mu, onun diliyle konuşmak gerekecek mi, ona veri nasıl sağlanacak, ürettiği çıktılar nasıl entegre edilecek?
- **Arayüz teknolojileri neler olacak?** Mobil arayüz olacak mı? Olacaksa iOS mu, Android mi, başka platformlar da desteklenecek mi? Hangi sürüme kadar geriye dönük destek verilecek? Native mi olacak, cross platform mu? Web arayüzü, masaüstü arayüzü, tablet gibi başka ortamlar olacak mı, olacaksa nasıl?
- **Diğer sistemlerle haberleşme nasıl olacak?** Hangi arayüzler, hangi teknolojiler, hangi protokoller ve hangi güvenlik yöntemi kullanılacak?

> **Ek bilgi (derste anlatılmadı):** Native uygulama, doğrudan tek bir platform için onun kendi araçlarıyla geliştirilen uygulamadır. Cross platform geliştirmede ise tek bir kod tabanından birden çok platform için uygulama üretilir.

Hocanın deyişiyle soruların sonu yoktur. Tasarım, bir çözüm adayı oluşturma aşamasıdır ve karar verilmesi gereken çok fazla nokta vardır.

> **Hocanın vurgusu:** Bu noktalar ne kadar detaylandırılırsa tasarım o kadar iyi olur. Üzerinde durulmayıp açık bırakılan her yer tasarımı o kadar muğlak ve kötü yapar.

8. Mimari Tasarım Bileşenleri

Mimari tasarım bileşenleri temelde iki parçaya ayrılır: donanım ve yazılım.

8.1 Donanım tarafı

Donanım tarafında istemci, sunucu ve ağ altyapısının nasıl olacağı belirlenir.

İstemci tarafı kullanıcıları ve erişim noktalarını ifade eder. Bu bir Amazon Kindle, akıllı telefon, akıllı saat olabileceği gibi bambaşka bir veri giriş ortamı, örneğin bir sensör ya da kamera da

olabilir. Hepsinin mimari tasarımının sistem modeline eklenmesi gerekir.

Sunucu tarafı iş yapan, gereksinimleri yerine getiren ve yazılımın durduğu birimlerdir. Dağıtık mı merkezi mi olacağı burada kararlaştırılır.

Ağ altyapısı bunların hangi ortamdan, hangi protokollerle haberleşeceğini belirler. Fizibilitede de bu tartışılmıştı: sistem önerisi kabaca şu kadar trafik yaratır, bu yüzden şöyle bir ağ altyapısı gerekir; ama kullanıcı sayısı 10.000 değil de 100.000 olursa şunlar artırılmalı ya da değiştirilmelidir. Yalnız geliştirme için değil, kullanım aşaması için 5 yıllık projeksiyonla birlikte düşünülmüştü. Tasarımda bu öneriler alınır ve gerçek hale getirilir.

8.2 Yazılım tarafı

Yazılım günümüzde genellikle katmanlara ayrılarak tasarlanır. Her katman diğerine hizmet verir ve ondan hizmet alır. Böylece problem alt parçalara ayrılır ve her parça kendi içinde daha küçük bir alt problem olarak daha kolay çözülür. Dikkat edilmesi gereken temel noktalar şunlardır:

- **Sunum katmanı:** İstemcilerle ve diğer sistemlerle haberleşir. Uygulama ile ön yüzler arasındaki kontrol ara tabakasıdır. Ön yüzler de bununla birlikte tasarlanır.
- **Uygulama katmanı:** Uygulama mantığının çözüldüğü, yani yazılım tasarımının yapıldığı katmandır.
- **Veri erişimi ve veri depolama:** Veriye nasıl erişileceği, yani veri erişim mantığı, ve verinin depolanması ile veri tasarımı burada ele alınır.

Donanım tarafıyla eşleştirildiğinde ön yüzler istemcide, uygulama mantığı ve veri katmanı sunucu tarafında yer alır. Aralarındaki bütün haberleşmeler ağ altyapısını oluşturur. Bunların hepsi tasarlanmalıdır.

9. Envanter, İş Yükü, Değerlendirme ve Satıcı Seçimi

Diyelim ki tasarlanacak bir sistem var. Bazı parçaları geliştirmek gerekiyor, bazılarını hazır alabiliriz. Elimizde iyi kötü çalışan bir bilgi sistemi ve belli miktarda donanım, örneğin uygulama ve veri tabanı sunucularının durduğu cihazlar var. Hoca altyapı kararlarının mantığını bu durum üzerinden sıraladı:

1. Elde ne olduğu, yani envanter belirlenir.
2. İş yükü tahmin edilir: mevcut donanım ve yazılım yeni sistemi karşılar mı?
3. Karşılıyorsa yeni altyapı yatırımı yapmadan devam edilir. Karşılamıyorsa yeni ilaveler ya da daha kapsamlı değişiklikler gerekir.
4. Bir şeyler alınacaksa kimden alınacağı belirlenir.
5. Finansal olarak nasıl alınacağı belirlenir: satın alma, leasing ya da kiralama.

9.1 Envanterin belirlenmesi

Bu adımda sistem analistleri yine rol alır. Mevcut sistemin altyapısına ve durumuna fizibilitedekinden daha ayrıntılı bakılır. Elde ne var, modeli ne, üreticisi kim, durumu ve yaşı ne, daha ne kadar çalışır, fiziksel olarak nerede duruyor, kim sorumlu? Finansal yapısı da sorulur: işletmenin malı mı, kiralık mı? Kiralıksa belki kira süresi dolmuştur ve devam etmemek gerekir. Sonra gerekli altyapı ile mevcut donanım kıyaslanarak alınması gereken kısım ortaya çıkarılır.

9.2 İş yükü tahmini

Burada kapasitenin yetip yetmediğine ve sistemin istenen zamanda tepki verip vermeyeceğine bakılır. Asıl amaç gereksiz satın alma ve sistem kurulumlarını mümkün olduğunca engellemektir. Ne kadar ilave iş yükü oluşacağı ve mevcut altyapının bunu kaldırıp kaldıramayacağı ortaya çıkarılır.

9.3 Donanım değerlendirmesi

Projenin ihtiyaçları ve tahmin edilen iş yükü ile mevcut envanter karşılaştırılır. Kullanılabilecek ölçütlere örnekler şunlardır. Bunlar yalnız örnektir, sayıları çok artırılabilir:

- Bir işlem için gereken ortalama süre: input ve process ile birlikte output üretme süresi beklentilerin içinde mi? İşlem örneğin 10 milisaniyede yapılıyor mu?
- Sistemin toplam kapasitesi: 1000 öğrenciye kadar mı, 10.000 öğrenciye kadar mı tepki verebiliyor?
- Ne kadar CPU atıl kalıyor?
- Ne kadar bellek ihtiyacı oluşuyor?

9.4 Yazılım değerlendirmesi

Elimizdeki yazılımlar şu ölçütlerle değerlendirilebilir: istekleri yerine getiriyor mu, ekranlar ve çıktılar uygun mu, kapasitesi ve performansı yeterli mi, kullanımı kolay mı, değişiklik kolay yapılabiliyor mu, yani esnek mi, dokümantasyonu iyi mi, üreticisi destek veriyor mu?

Sonuçta ne kadarının kullanılacağı, ne kadarının yeniden yapılacağı ya da alınacağı ön tasarımda ortaya konmaya çalışılır. Hocanın deyişiyle mümkün olduğunca az iş yapmaya çalışılır.

9.5 Satıcı seçimi

Bir şey satın almak gerekiyorsa nereden alınacağına satıcının verdiği desteklere bakılarak karar verilir: donanım desteği, yazılım desteği, kuruluş ve eğitim hizmetleri ve bakım aşamasındaki destek. Bakım desteği sözleşmeye ve ödenen paraya göre seviyelere ayrılır:

- Yalnız hafta içi mesai saatlerinde destek.
- Akşamların dahil edildiği, resmi tatillerin hariç olduğu destek.
- 7 gün 24 saat, istendiği anda destek. Bu çok daha pahalıya mal olur.

10. Edinme Biçimi: Satın Alma, Leasing, Kiralama

Bu karar biraz da üst yönetimle, yani patron tarafıyla birlikte verilir. Hoca konuyu fiziksel bir sunucu üzerinden anlattı: önceki aşamalardan sonra, performans, işlem hacmi ve gereksinimler için şu özelliklerde bir cihazın alınması gerektiği ortaya çıktı. Peki bu nasıl edinilecek?

Satın alma. Uzun dönemde daha ucuza gelir. İstenirse ölçeklendirilebilir. Tamamı sizindir ve tam kontrol sizdedir. Amortismanla gider yazılarak vergi indiriminden de yararlanılabilir. Dezavantajları da vardır. Para baştan ödenir, taksitle bile olsa baştan bağlanır. Teknolojinin değişim hızı düşünülürse cihaz çok çabuk eskiyebilir. Fizibilitedeki eksiklerden ya da tasarımdaki yanlış kararlardan ötürü doğru cihaza yatırım yapılmamış olabilir; yani yatırımın batma riski vardır. Cihazla ilgili bütün sorumluluk da sizdedir.

> **Ek bilgi (derste anlatılmadı):** Amortisman, uzun süre kullanılan bir varlığın maliyetinin kullanım ömrüne yayılarak dönem dönem gider yazılmasıdır.
>
> **Ders sonrası not:** Hoca bu avantajı “gelir vergisinin azaltılması” diye anlattı. Amortisman vergiyi doğrudan değil, gider olarak vergi matrahını azaltarak düşürür. Şirketin türüne göre söz konusu vergi büyük olasılıkla kurumlar vergisi de olabilir.

Finansal kiralama (leasing). Parayı doğrudan cebinizden çıkarmazsınız. Banka gibi bir kurum sizin için parayı öder, siz de kira öder gibi ödersiniz. Normalde kiradan daha düşük bir bedel ödenir ve büyük bir para bağlamak gerekmez. Ama kurum kendi kârını ve riskini üstüne koyduğu için leasing genelde satın almadan pahalı, kiralamadan ucuzdur. Ödemede sıkıntı yaşanırsa cihaz elinizden gidebilir ya da daha yüksek faizle daha fazla ödemek zorunda kalabilirsiniz. Finansal durumunuz elveriyorsa ve bir risk öngörmüyorsanız, bütün parayı bir arada çıkarmadığınız için mantıklı bir seçim olabilir.

Kiralama. Ürün sizin değildir, yalnız bir süre için kiralayıp kullanırsınız. Toplam kullanım ömründe satın almadan ya da leasingden çok daha fazla para harcarsınız. Buna karşılık bakım ve garanti genellikle kiralama hizmetiyle birlikte gelir. Ekstra finansmana da gerek yoktur, her ay zamanı geldikçe parça parça ödersiniz. Son kullanıcılardan abonelikle aylık gelir toplayan bir yapınız varsa, siz de altyapıyı kiraladığınız yere aylık ödeme yapabilirsiniz. Sıkıldığınızda “tamam, bitiriyorum” deyip çıkarsınız ve elden çıkarmanız gereken bir sunucu ya da donanım kalmaz.

Biçim	Toplam maliyet	Sahiplik	Öne çıkan artı	Öne çıkan risk
Satın alma	Uzun dönemde en düşük	Tamamen sizin	Tam kontrol, amortisman	Baştan para, eskime, batma riski, bütün sorumluluk
Leasing	Satın almadan yüksek, kiralamadan düşük	Satın alınmış olur, ödemeler kira gibi	Baştan büyük para gerekmez	Ödeme aksarsa cihaz kaybı ya da yüksek faiz
Kiralama	En yüksek	Sizin olmaz	Bakım ve garanti genelde dahil, kolay çıkış	Toplamda çok daha fazla ödeme

Hangi biçimin seçileceği işe, probleme ve sistemin nasıl çalışacağına göre değişir.

11. Ayrıntılı Tasarım ve Çıktı Tasarımı

Ön tasarımın bir parçası olan modüler çözümleme, ayrıntılı tasarımdaki uygulama tasarımına çok yakın olduğu için onunla birlikte anlatılacaktır: ilk kısmı ön tasarımda yapılır, kalanı ayrıntılı tasarımda bitirilir. Bütün program tasarımını tek adımda anlatmak daha doğrudur.

Ayrıntılı tasarımda beş ana tasarım aşaması vardır: çıktı tasarımı, girdi tasarımı, ara birim tasarımı, uygulama ya da program tasarımı ve veri tabanı tasarımı.

Bir bilgi sisteminin çıktılarını tasarlamak, kullanıcılara verilen bilginin ya da üretilen raporun nasıl olacağına karar vermektir. Tasarımdaki ana soru hep “nasıl?” sorusudur: Bu rapor nasıl oluşturulacak, bu bilgi nasıl üretilecek, formatı ve ortamı nasıl olacak?

Çıktı tasarlarken dikkat edilecek noktalar şunlardır:

- **Amaç ve anlam:** Çıktı amacına hizmet etmeli, kullanıcı için anlamlı olmalı, alakasız içerik barındırmamalıdır.
- **Sayı:** Uygun sayıda çıktı üretilmelidir.
- **Kullanıcı:** Çıktının hangi kullanıcılara dağıtılacağı, kimlerin erişme ve kullanma hakkı olduğu doğru belirlenmelidir.
- **Zaman:** Çıktıda zaman ekseni varsa çıktı buna uygun, örneğin günlük, aylık ya da yıllık üretilmelidir.
- **Ortam:** Çıktı doğru ortamda üretilmelidir. Ekrana mı yansıtılacak, bir billboardda ya da duvar ekranında infografik olarak mı duracak? Stoktaki çalışanın eline kağıt liste olarak mı, kargodaki kamyonun şoförüne kağıtla mı verilecek? Yoksa birinin akıllı saatine uyarı olarak mı gidecek?
- **Sadelik:** Çıktılar temiz, düzgün ve yalın olmalıdır.

Kısaca her çıktı için hangi ortamda, hangi yöntemle, hangi zamanda, hangi içerikle ve kimi hedefleyerek üretileceği sorulur.

Ekrandaki raporlar ve listeler için hoca şu beklentileri ekledi. Liste sıralanabilmelidir, örneğin kolona tıklanınca büyükten küçüğe sıralanmalıdır. Basit filtre uygulanabilmelidir: “binden büyük olanlar kalsın, gerisini gösterme” gibi. Sınırlar belli olmalıdır: aylık bir çıktıysa bir ay, ama bu aralık değiştirilebilir olmalıdır. Grafik çıktılarda grafiğin tipi ve renkleri düzgün seçilmeli, grafik ölçeklenebilir olmalı ya da tipler birbiriyle değiştirilebilmelidir.

Çıktı tasarımı için “şöyle yapın” denebilecek tek bir reçete yoktur. Yalnız bu noktalara dikkat edilmesi istenir.

12. Girdi Tasarımı

12.1 Kaliteli girdi için ilkeler

Benzer kurallar girdi tasarımı için de geçerlidir. Amaç, kaliteli girdi sağlanmasına öncülük etmektir.

Bütünlük. Bir form ya da ekrandaki dizilimin bir amacı olmalı, daldan dala atlanmamalıdır. Bir ekranda kişisel bilgiler alınıyorsa orada eğitim bilgileri yer almamalıdır. İkinci ekranda eğitim bilgileri alınıyorsa orada alakasız şekilde hobiler sorulmamalıdır.

Yeterlilik ve doğruluk. Analizde tanımlanan bütün işlemleri yerine getirmek için gereken veriler eksiksiz alınmalı, kontrol edilmeli ve doğruluğu sağlanmalıdır.

Ergonomi. Yıl girdisi alınıyorsa yıl zaten büyük olasılıkla "20" ile başlar. Kısa bir aralıktan yıl alınıyorsa son haneyi yazmak bile yeterli olabilir. Dört tuşa basmak yerine iki ya da üç tuşa basmak tercih edilir. Benzer şekilde 31 günün içinden kaydırma çubuğuyla gün seçmek ergonomik olmayabilir: kaydırırken el kayabilir, yanlış bir değer seçilebilir.

Uyumluluk. Kişisel bilgileri aldınız, ileri düğmesine basıldı, ikinci ekranda eğitim bilgileri alınıyor. Bir anda fontlar değişmiş, girdiler soldan sağa gitmiş, metin kutularının yerine bambaşka öğeler gelmişse bu kötüdür. Ekranlar arasında düzen değişmemelidir.

Basitlik ve çekicilik. Ekran ve formlar basit, sade ve insana itici değil çekici gelecek şekilde tasarlanmalıdır. Hoca bunun tek bir doğrusu olmadığını, ama farkın hissedildiğini söyledi. Bazı uygulamalarda baş parmakla bir şeyleri çekiştirerek pek çok işlev yerine getirilir. Bazılarında ise önce bir yere tıklamak, onu etkinleştirmek, klavyenin açılmasını beklemek ve uzun uzun yazmak gerekir. İkincisi kötü girdi tasarımıdır.

12.2 Kötü ve iyi girdi örneği: mobilde silme

Hoca, öğrencilerin ileride geliştireceği mobil arayüzlü bilgi sistemlerinde sık gördüğü bir kötü örnek verdi. Bir listeden eleman silmek için üstten işlemler menüsü açılır, silme seçilir, silinecek eleman seçilir, yanında bir işaret kutusu çıkar, o işaretlenir, aşağıda OK düğmesine basılır. Neden böyle yapılır? Çünkü bir düğme ve işaret kutusu koyup tıklanınca etkinleştirmek gibi bir tasarımı kodlamak çok kolaydır.

Olması gereken ise kullanım kolaylığıdır. Mobil arayüzde satır baş parmakla sağa kaydırıldığında bir silme düğmesi çıkmalı ve doğrudan tıklanarak silinebilmelidir. Basılı tutup uzun kaydırınca otomatik silinmesi ya da yalnız bir onay sorulması da sağlanabilir.

Basitlik, çekicilik, ergonomi ve kullanım kolaylığı bu yüzden önemlidir. Bazı uygulamaları sevip "ne kadar kolay" derken, bazılarını takvime tek bir şey yazmak için 50 yere bastığınız, bir şey unutunca hata verip başa döndürdüğü için silersiniz. Kullanıcı kabulü için sosyal fizibilitede de buna değinilmesi gerekir: Uygulamayı yaşlılar mı, gençler mi, küçük çocuklar mı kullanacak? Girdiler bu kişilerin sağlayabileceği biçimde tasarlanmalıdır.

12.3 Toplu girdi, kaynakta veri, az tuşlama

Sistem elle girdinin yanı sıra toplu girdiye (batch ya da bulk input) de olanak vermelidir. Hoca OBS'den örnek verdi: vize sınavı ya da ödev notlarını sisteme girerken her öğrencinin karşısına tıklayıp tek tek yazmak gerekiyorsa bu kötü bir giriş şeklidir. Oysa notlar okunurken zaten Excel'e yazılmıştır. Excel'den tek seferde içe aktarılabilse çok daha kolay bir girdi sağlanmış olur.

Veri mümkün olduğunca kaynağında tutulmalı, başka yerlerde tutulmamalıdır. Tıklama ve tuşlama sayısı olabildiğince az olmalıdır.

12.4 Doğrulama ve geçerleme: dört temel kontrol

Bir veri alınırken doğrulama ve geçerleme işlemleri her zaman yapılmalıdır.

> **Ders sonrası not:** Hoca "doğrulama ve geçerleme, yani validation and verification" dedi; iki kelimenin sırası Türkçe karşılıklarıyla ters düşüyor. Doğrulama "verification", geçerleme "validation" karşılığıdır.
>
> **Hocanın vurgusu:** Girdide dört ana kontrol mutlaka yapılmalıdır: veri tam mı, biçimi doğru mu, aralığı doğru mu, tutarlı mı? Bu kontrolleri yaparsanız iyi tasarım, yapmazsanız kötü tasarım yapmış olursunuz.

Hoca dört kontrolü tarih örnekleriyle anlattı. Tarih seçilmesinin sebebi basit olmasıdır; aynı mantık pek çok veriye uygulanır.

Tamlık. “Doğum tarihinizi giriniz” alanına kullanıcı yalnız 1999 yazıp geçti. Bu veri tam değildir, çünkü sistemin çalışması için belki gün ve ay da gerekir. Kullanıcı bunları girmeye yönlendirilmeli ya da bunlar girilmeden bir sonraki adım etkinleştirilmemelidir.

Biçim. Kullanıcı gün, ay, yıl biçiminde mi, ay, gün, yıl biçiminde mi giriyor? Amerikalılar ay, gün, yıl biçimine alışmıştır. Amerika’daki kullanıcılar için yazılan bir sistemde girdinin bu biçimde alınması ve kontrol edilmesi gerekir.

Aralık. Doğum tarihine 1899 yazıldığını düşünelim. Hesap şöyledir:

- Verilen: doğum yılı 1899, kayıt yılı 2023.
- İşlem (ara adım eklendi): $2023 - 1899 = 124$.
- Sonuç: kullanıcı 124 yaşında görünür. Çok büyük ihtimalle yanlış yazılmıştır, çünkü 1800’lerde doğmuş olamaz.

Başka bir örnek: çocuğun doğum tarihine 2022 yazılacakken yanlışlıkla 2122 ya da 0 ile 9 tuşları yan yana olduğu için 2922 yazılıyor. Bu kez daha gelmemiş bir tarih girilmiş olur. Aralık yine yanlıştır.

Tutarlılık. Doğum yılı 1999 girildi. İlk işe başlama tarihinde ise 2023 yerine yanlışlıkla 2003 yazıldı. Hesap şöyledir:

- Verilen: doğum yılı 1999, işe başlama yılı 2003.
- İşlem (ara adım eklendi): $2003 - 1999 = 4$.
- Sonuç: kullanıcı 4 yaşında işe başlamış görünür. İki değer tek tek aralık içinde olabilir, ama birbiriyle tutarlı değildir.

Benzer şekilde, doğum tarihi ile evlilik tarihi karşılaştırıldığında kişi doğmadan önce ya da 10 yaşında evlenmiş görünüyorsa bu da bir tutarsızlıktır.

Kontrol	Sorusu	Örnek hata
Tamlık	Veri eksik mi?	Yalnız yıl girildi: 1999
Biçim	Beklenen biçimde mi?	Gün-ay-yıl mı, ay-gün-yıl mı?
Aralık	Geçerli sınırlar içinde mi?	1899 (124 yaş), 2122 ya da 2922 (gelecek)
Tutarlılık	Diğer verilerle uyuşuyor mu?	Doğum 1999, işe giriş 2003 (4 yaş)

Kolay girdi tasarımının güncel bir örneği de mobil ortamlarda oturum açmadır. Artık kimse bir şey yazmak istemiyor, Facebook ya da Google ile oturum açıp devam etmek istiyor. Bu da kullanım kolaylığını hedefleyen bir girdi tasarımıdır.

12.5 Ekran tasarımı

Girdi alınan ekranlar için hoca şunları sıraladı:

- Ekran kolay kullanımlı ve basit olmalı, gereksiz bilgi barındırmamalıdır.
- Pencereleme ya da kutulama yapısı kullanılmalı, kutuların içinde gerekli açıklamalar verilmelidir: girdinin hangi formatta olacağı ve özel yönlendirmeler gibi.
- Ekranlar arasında tasarım uyumu olmalıdır.
- Hareket kolaylığı olmalıdır: ekrandan ekrana geçiş, kolonu tutup kaydırma, tıklayınca sıralama gibi.
- Belli bir düzen olmalı, renkler, yazı tipleri, imleç ve boyutlar sürekli değişmemelidir.

Günümüzde bunların çoğu web tabanlı olduğundan ve CSS tabanlı biçimlendirmeyle yapıldığından bu işler çok daha kolay hale gelmiştir.

13. Ara Birim Tasarımı

13.1 Ara birim türleri

Kullanıcıyla etkileşim yalnız ekranla olmaz. Sistemde varsa şu ara birimlerin de tasarlanması gerekebilir:

- Doğal dil ara birimleri: ChatGPT gibi bir şey sorup cevabını aldığınız arayüzler.
- Soru-cevap biçiminde sorgulama.
- Menü ara birimleri.
- Çeşitli formlar.
- Uygulamaya özel bir script ya da komut dili.
- Sistem bildirimleri için ara birim.

Kullanıcıyla iletişime girilen her kısım düzgün tasarlanmalıdır. Söylenecekler girdi tasarımından çok farklı değildir: etkinlik ve verimlilik önemlidir, kullanıcı doğru girdileri sağlayabilmeli, ara birim ergonomik olmalıdır.

13.2 Ekranın üç bölümü

Ekran arayüzleri genellikle üç parçaya bölünür:

- **Gezinim alanı:** İşlevler arasında gezinmeyi sağlar. Word'ün en üstündeki dosya, düzenleme, yazı tipi ve tablo düğmeleri gibi üstte olabilir ya da OBS'de olduğu gibi solda bir kolon olabilir.
- **Orta alan:** En büyük alandır. Kullanıcının yaptığı çalışmalara, yani ana ekrana ayrılır.
- **Durum alanı:** En altta, yapılanlarla ilgili durumun ve daha teknik bilgilerin gösterildiği status bar yer alabilir.

13.3 Sık görülen tasarım problemleri

Bugüne kadar tasarlanmış sistemlerde ara birimlerle ilgili genel problemler şunlardır:

- Bilgi düzensiz, yığınlar halinde görünebilir.
- Geçişler uyumsuz olabilir.
- Terminoloji karmaşık olabilir.
- Hata mesajları saçma ya da açıklamasız olabilir. "Bilinmeyen bir durum oluştu" gibi bir mesaj hiçbir bilgi vermez. Hatanın neden oluştuğu daha net açıklanmalıdır.
- Kullanıcı kılavuzları ve el kitapları anlaşılmaz derecede karmaşık olabilir.

Girdi, çıktı ve ara birim tasarımında "oturun şöyle yapın" denemez. Yalnız nelere dikkat edileceği söylenebilir.

> **Hocanın vurgusu:** "Kullanıcının aptal olduğunu her zaman unutmayın." Bütün doğrulama, geçerleme ve kontroller yapılmalıdır. "Framework onları zaten kontrol ediyor" diyebilirsiniz, ama buna rağmen hiçbir girdiyi kontrol etmeden sisteminize almayın, aldığınız haliyle doğrudan kullanmayın. En basitinden bir SQL injection saldırısı bile bu kontrollerin eksikliğinden doğar. Yani girdi tasarımının düzgün yapılmaması bir güvenlik açığı bile yaratabilir.
>
> **Ek bilgi (derste anlatılmadı):** SQL injection, kullanıcıdan alınan metnin kontrol edilmeden bir veri tabanı sorgusunun parçası yapılması sonucu, saldırganın o metin üzerinden sorgunun anlamını değiştirebilmesidir. Bu yüzden hem girdi denetimi hem de girdiyi sorgu metninden ayıran yöntemler kullanılır.

Soru: Bir öğrenci, Windows'un disk biçimlendirme sırasında bazen yalnız "bir şey oldu" deyip işlemi sonlandırdığını söyledi. **Cevap:** Hoca bunu şaşkınlıkla karşıladı ve "Hiçbir şey olmasa bile 'bir şey oldu' dese daha da garip olurdu" diye takıldı. Örnek, açıklamasız hata mesajı problemini gösterir.

14. Yazılım Tasarımı ve Modüler Program Yapısı

Girdi, çıktı ve ara birim başlıkları geçildi. Geriye iki ana kısım kalır: yazılımın mimarisinin ortaya çıkarılması, yani yazılım tasarımı, ve veri tabanı tasarımı. Veri tabanı tasarımı daha sonra ayrı bir haftada ele alınacaktır. Bu bölümün sorusu şudur: işlevsel çözümlemeyle, yani veri akış diyagramlarıyla yola çıkıldığında yazılımın mimarisi nasıl belirlenir ve tasarım nasıl ifade edilir?

Ön tasarımda modüler yapı belirlenir, ayrıntılı tasarımda modüllerin neler olması gerektiği ve iç birimlerin nasıl çalışacağı anlatılır. Derste ikisi birlikte işlendi.

Modüler program yapısı, veri akış diyagramlarındaki işlevleri alıp çalışan yazılım birimlerine, parçalarına ve modüllerine dönüştürmek demektir. Aralarındaki hiyerarşi ve ilişkiler ortaya konur, varsa kontrol mekanizmaları kabaca belirlenir ve hepsi grafik olarak ifade edilir. Bu yapı programı verilerle birleştirir, yani hangi parçanın hangi veriyle çalışacağını gösterir. Varsa ara birimler de tanımlanır.

Bunun için temel olarak bir diyagram kullanılır: **yapı diyagramı** (structure diagram). Hoca bu diyagramın çok basit olduğunu, tıpkı veri akış diyagramında olduğu gibi dikkat edilecek noktalarıyla anlatılacağını söyledi.

15. Yapı Diyagramının Öğeleri

Yapı diyagramının içinde şu öğeler bulunur:

Öğe	Gösterim	Anlamı
Modül	Dikdörtgen	Anlaşılması ve bakımı kolay mantıksal program işlem birimi
Kontrol modülü ve alt modül	Üstte kontrol modülü, altta alt modüller	Üstteki, alttakilerin çalışmasını kontrol eder
Kütüphane modülü	Çift çizgili kenarlı dikdörtgen	Hazır kullanılan, içe aktarılan modül
Veri mesajı	İçi boş halkalı ok	Modüller arasında iletilen veri
Kontrol mesajı	İçi dolu halkalı ok	Bir durumun ya da olayın bildirilmesi
Koşul	Eşkenar dörtgen (baklava deseni)	Koşula bağlı modül çağrımı
Döngü	Alt modüllerin üzerinde sağdan sola eğri ok	Alt modüller döngü içinde çalışır

15.1 Modüller ve hiyerarşi

Modül, veri akış diyagramlarındaki işlem kutucuklarının yazılım uzayına taşınmış hali gibi düşünülebilir. Anlaşılması ve bakımı kolay olsun diye gerektiğinde birkaç parçaya ayrılabilir.

Modüller hiyerarşik yerleştirilir. Slayttaki genel gösterimde Modül 1, altındaki Modül 1.1, 1.2 ve 1.3'ün çalışmasını kontrol ettiği için **kontrol modülü** adını alır. Modül 1 çalışırken isterse 1.1'i, isterse 1.2'yi, isterse 1.3'ü çalıştırabilir. Alt modüller aynı anda çalışmaz; bunlar seçeneklerdir. Örneğin bir ödeme modülünün altında nakit ödeme ve kredi kartıyla ödeme alt modülleri olabilir.

Bir modül hazır bir kütüphane modülünü de kullanabilir. Bu tür modüller **kütüphane modülü** (library) olarak kenarlarına çift çizgi çizilerek gösterilir. Örneğin bir form oluşturmak için kullanılan form kütüphanesi kontrol modülüne dahil olacak şekilde çizilebilir.

15.2 Veri ve kontrol mesajları

Veri akış diyagramlarında her şey akan verilerden oluşuyordu ve oklar çiziliyordu. Yapı diyagramında da modüller arasında hangi verilerin iletildiği gösterilir. Bunlar veri olabileceği gibi kontrol verisi de olabilir. Buna **kontrol mesajı** denir. Bir durumun ya da hareketin aktarılması için kullanılan özel mesajdır. Örneğin dosya okunurken dosyanın sonuna gelinmesi bir olaydır: "dur, dosyanın sonu geldi" mesajı yapı diyagramında kontrol mesajı olarak gösterilebilir.

Hoca kontrol mesajını exception'a benzetti. Bir kontrol mesajı görüldüğünde onu ele almak gerekir. Ön tasarımda bunu göstermek, "ayrıntılı tasarımda bunu lütfen yapın" demek anlamına gelir.

15.3 Koşul ve döngü

Eşkenar dörtgen koşula bağlı çağrıyı gösterir: koşula göre alt modüllerden biri çalışır. Eğri ok ise alt modüllerin döngü içinde çalıştığını gösterir. Koşulun kendisi üst modülün akışında ya da algoritmasında ayrıca tasarlanır.

16. Yapı Diyagramı Örnekleri

Aşağıdaki modül adları slaytlardaki İngilizce adlardır. Altyazıda bir kısmı Türkçe anlatıldığı için adlar kısa nottaki slayt bilgisiyle eşleştirildi.

16.1 Müşteri verisinin bakımı

```
Maintain Customer Data
|-- Check Customer Status
`-- Update Customer File
```

Üstte müşteri verisinin bakımıyla ilgili bir modül vardır. İçinde ne olduğunu, algoritmasını ve kodunu bu diyagramdan göremeyiz; çünkü bu aslında bir ön tasarımdır. Altında iki alt modül tanımlanmıştır ve üst modül bunlardan birini çalıştırabilir.

Çağrılan alt modül	Giden	Dönen	Tür
Check Customer Status	Customer Number	Customer Status	Veri, veri
Update Customer File	Customer Charges	Account Overdue	Veri, kontrol mesajı

Soldaki modülün çalışması için ona mutlaka müşteri numarası verilmelidir. Yani numara bu modülün parametresidir; numara olmadan çalışmaz. Modül çalışır ve karşılığında müşterinin durum bilgisini üretir. Üst modül artık bu çıktıya sahiptir. Duruma göre sağdaki modül çalıştırılabilir ya da çalıştırılmayabilir; diyagramda bununla ilgili bir ayrıntı yoktur.

Güncelleme modülüne ise harcanan tutar bilgisi gönderilir. Modül çalışır ve hiçbir şey döndürmeyebilir ya da bir kontrol mesajı döndürebilir. Üst modülde bununla ne yapılacağına dair ayrıntı diyagramda yoktur, ama böyle bir mesajın varlığı şunu söyler: bu özel durum üst modülün algoritmasında ele alınmalıdır. Hoca bu veri ve kontrol mesajı düzenini **kontrol çifti** diye adlandırdı.

> **Ders sonrası not:** Altyazıda “Account over” geçiyor ve hoca mesajı “hesap bakiyesi yetersiz” ya da “hesap taştı” diye açıkladı. Kısa nottaki slayt adı Account Overdue’dur. “Overdue” vadesi geçmiş demektir; yani mesaj müşterinin ödemesinin vadesinin geçtiğini bildirir.

> **Ders sonrası not:** Hoca “kontrol çifti” terimini veri gönderme ile kontrol mesajı alma ikilisi gibi anlattı. Slayttaki “Control Couple” teriminde “couple” büyük olasılıkla ikiliyi değil, modüller arasındaki bağı anlatır; yani terim büyük olasılıkla tek bir kontrol mesajını adlandırır.

16.2 Koşula bağlı çağrı: parça sıralama

```
Sort Inventory Parts  <koşul>
|-- Sort By Part Name
|-- Sort By Part Number
`-- Sort By Inventory Value
```

Sıralama modülü ada, numaraya ya da envanter değerine göre sıralayabilir. Her biri başka işlemler gerektirdiği için ayrı bir alt modüldür. Baklava deseni, koşula göre bunlardan birinin çalışacağını gösterir. Koşulun kendisi üst modülün akışında tasarlanır.

16.3 Döngü: not raporu

```
Produce Grade Reports
|-- Get Student Grades   (döngü)
|-- Calculate GPA        (döngü)
`-- Produce Grade Report
```

Burada sağdan sola doğru bir eğri ok görülür. Hocanın açıklamasıyla bu alt modüller tek başlarına çalışmaz; ok altındaki ikisi birlikte, sırayla, döngü halinde çalışır. Akış şöyledir:

1. Üst modül Get Student Grades modülüne öğrenci numarasını verir: "şu numaralı öğrencinin notlarını ver."
2. Bu alt modül veri tabanından sorgularla notları okur ve Grades verisini döndürür. Bununla birlikte, kayıt kalmadığını bildiren bir End of File kontrol mesajı da dönebilir. Üst modül bunu ele almalıdır.
3. Üst modül notları Calculate GPA modülüne verir, o da ağırlıklı not ortalamasını hesaplayıp GPA olarak döndürür.
4. Sonuç ekranda gösterilebilir ya da GPA listesi Produce Grade Report modülüne verilip rapor üretilebilir. Bu modül de üretilen Grade Report verisini döndürür.

Diyagramdan raporun ekranda mı kağıtta mı olduğu görülmez. Yalnız böyle bir rapor verisinin oluşturulup döndürülmesi gerektiği görülür. Hocanın deyişiyle yapı diyagramı "bu modülün çalışması için bunlar gerekir, çalışınca şu çıktıyı üretir ve sana verir" ifadelerini gösterir.

16.4 Bordro üretimi

```
Generate Payroll
|-- Get Payroll Record
|   |-- Read Payroll Record
|   `-- Validate Payroll Record
|-- Calculate Net Pay
|   |-- Calculate Gross Pay
|   |-- Calculate Deductions
|   |   |-- Calculate Tax Withheld
|   |   `-- Calculate SS Withheld
|   `-- Update Employee Record
`-- Print Check
```

Bu modül bordro ya da ödeme makbuzu üretir. Tepesinde başka bir modül vardır: kullanıcı menüden "bordro oluştur" dediğinde bu modülün çalışması istenir. Çalışması için ona veri vermek gerekmez. İş şu sırayla yürür:

1. **Kayıtların alınması.** Get Payroll Record içinde okuma ve geçerleme vardır. Okuma alt modülü okunan kaydı ya da "bitti" bilgisini döndürür. Geçerleme alt modülüne kayıt verilir ve "geçerli mi?" diye sorulur; geçerli ya da değil diye bir kontrol mesajı döner. Sonunda üst modüle kayıtlar ya da artık kayıt kalmadığını bildiren bir kontrol mesajı döner.
2. **Net ödemenin hesaplanması.** "Bitti" mesajı geldiğinde net hesaba geçilir. Kayıtlar verilir; önce brüt ödeme, sonra tenzilatlar ve indirimler hesaplanır, ardından kayıt güncellenir. Bunların da altında çalışacak alt modüller vardır.
3. **Makbuzun basılması.** Son olarak "bunları yazdır" denir. Print Check bunları kağıda basar ve bastığını bildiren Check Printed kontrol mesajını döndürür.

Veri akış diyagramının ikinci seviyesinde 1.1, 1.2 diye numaralandırılan işlemler, burada bir alt seviyedeki modüler çözümleme parçaları olarak yerleşir. İstenirse daha alt seviyeler de çizilebilir. Hoca, yapı diyagramındaki parçaların örnekleriyle birlikte kişisel olarak araştırılmasının faydalı olacağını söyledi.

17. Tasarım Belirtimleri

Altyapı kararları verilip modüler çözümleme yapı diyagramıyla ifade edildiğinde ön tasarım biter ve bir çıktı üretilmesi gerekir: **ön tasarım belirtimi** (spesifikasyonu). Burada genel özellikler, ilişkiler ve modüler çözümleme yer alır.

Ayrıntılı tasarımda ise yapı diyagramının bütün detayları, algoritmik ayrıntılar ve kontrol akışları **ayrıntılı tasarım belirtiminde** toplanır.

18. Modüllerin İçini Göstermek: IPO Kartları ve HIPO

Modüler çözümlemeden sonra modüllerin iç yapılarının tasarımı da ifade edilmelidir. Yani ayrıntılı tasarımda yazılım tasarımı nasıl gösterilecektir? Bunun için birkaç yaklaşım vardır, ama hoca bunların artık pek kullanılmadığını baştan söyledi.

Yapı diyagramındaki her modül ve alt modülün nasıl çalışacağını ifade etmek için **IPO kartları** (input, process, output) oluşturulabilir. Bu kartların hiyerarşisi modüler çözümlemeye eklenerek bir çözüm ortaya konabilir. Bunun için üç öğe hazırlanabilir:

- **Görsel içerik tablosu:** Kitabın içindekiler tablosu gibidir. Uygulamanın ana menüsü ya da ana modülü, onun alt modülleri ve onların alt modülleri gösterilir: bir menü açıp alt menüye, oradan bir alt menüye daha girmek gibi.
- **Genel IPO diyagramları:** Her fonksiyon için girdiler, prosesler ve çıktılar birer kartta yazılır.
- **Ayrıntılı IPO diyagramları:** Verilerin hangi dosyadan okunacağı, işlemin hangi adımına gireceği ve nereye kaydedileceği yazılır. Her adımın nasıl yapılacağı sözde kod (pseudocode) ya da akış diyagramıyla verilebilir.

  Ders sonrası not: Altyazıda “IPO” için “Hierarchy yani input, process, output” açılımı geçiyor. Hiyerarşiyi de içeren yöntemin adı HIPO’dur (Hierarchy plus Input-Process-Output). IPO ise yalnız Input-Process-Output demektir.

19. Bugün Uygulamada Ne Yapılıyor?

Hoca bu araçların güncel kullanımı konusunda açık konuştu. Yapı diyagramı bile artık çok kullanılmıyor. Sebebi şudur: İşlevsel çözümlemede çizilen veri akış diyagramları fiziksel veri akış diyagramına dönüştürülebilir ve dönüştürülen hal zaten fiziksel modülleri verir. Bu genellikle yeterli olur. Yine de modüler çözümlemeyi kontrol ve veri mesajlarıyla birlikte göstermek isterseniz yapı diyagramını kullanırsınız.

IPO tarafı ise hemen hiç yapılmaz. Hocanın deyişiyle “kimse yapmaz”. Modüler çözümlemeyi yapı diyagramıyla göstermek yeterlidir. İşlevsel çözümlemede genellikle veri akış diyagramlarının fiziksel hale getirilmesi de yeterli olur.

Günümüzde uygulamaların tasarımı genellikle nesneye yönelik çözümleme üzerinden yürür ve bu, **nesneye dayalı tasarım modelinin** oluşturulması demektir. Ayrıntılı tasarımdaki yazılım tasarımı adımında, nesneye yönelik çözümlemenin çalışan yazılım sınıfları halinde nasıl modelleneceği sonraki dersin konusudur.

Dersin Toparlaması

Bu derste “ne” sorusundan “nasıl” sorusuna geçildi. Sistem tasarımı ön tasarım ve ayrıntılı tasarım olarak iki safhaya ayrıldı ve bir bilgi sisteminin yalnız koddan ibaret olmadığı görüldü: kullanıcı arayüzü, sistem arayüzü, güvenlik, uygulama ve veri tabanı ayrı ayrı tasarlanmalıdır. Model güdümlü, hızlı uygulama geliştirme ve ortak uygulama geliştirme yaklaşımları karşılaştırıldı. Ön tasarımda çözümün temin yolu, işlevsel olmayan gereksinimlerin doğurduğu mimari sorular, envanter, iş yükü, satıcı seçimi ve edinme biçimi ele alındı.

Ayrıntılı tasarımda çıktı, girdi ve ara birim tasarımının ilkeleri ve girdinin dört temel kontrolü (tamlık, biçim, aralık, tutarlılık) işlendi. İşlevsel çözümlemeden gelen yazılım tasarımı için yapı diyagramı ve IPO kartları görüldü. Sonraki derste aynı soru nesneye yönelik taraf için sorulacak: nesneye yönelik çözümleme nasıl yazılım tasarımına dönüştürülür? Veri tabanı tasarımı ise ayrı bir haftada işlenecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.